# Control Flow Graphs (CFG)

## 1 Introduction

Program flow analysis studies how control and data move through a program.

**Program Flow Analysis** is the process of analyzing the execution structure of a program to understand how control and data propagate.

It can be classified as:

- **Control Flow Analysis** — determines the control structure and builds Control Flow Graphs (CFGs)
- **Data Flow Analysis** — determines how data values move and builds Data Flow Graphs (DFGs)

CFG construction assumes no information about actual data values — edges indicate paths that *may* be taken.

## 2 Control Flow Analysis

A **Control Flow Graph (CFG)** is a directed graph that represents all possible execution paths of a program.

CFGs are used to:

- Analyze program execution
- Detect unreachable code
- Understand loops and conditionals
- Optimize programs in compilers

## 3 Basic Blocks

A **basic block** is a sequence of statements that:

- Execute sequentially
- Have exactly one entry point
- Have exactly one exit point

**Basic block example**

```
a = 5
b = a + 2
c = b * 3
```

Control statements (`if`, `while`, `return`) are not basic blocks.

### 3.1 Formal Definition of a Basic Block

A **basic block** is a sequence of consecutive intermediate-language statements in which:

- Control can only enter at the beginning
- Control can only leave at the end

Only the **last statement** of a basic block can be a branch statement, and only the **first statement** can be the target of a branch.

## 4 Edges and Nodes

An edge from block $B_1$ to block $B_2$ indicates that execution of $B_2$ may immediately follow execution of $B_1$.

**Special nodes:**

- Entry node — start of execution
- Exit node — termination of program

A Control Flow Graph (CFG) is a directed graph where:

- Nodes represent basic blocks
- Edges represent possible control flow between blocks

The entry node corresponds to the basic block containing the first program statement.

## 5 Partitioning a Program into Basic Blocks

To construct a CFG, a program must first be partitioned into basic blocks.

### 5.1 Leader Statements

A **leader statement** is the first statement of a basic block.

Leader statements are identified using the following rules:

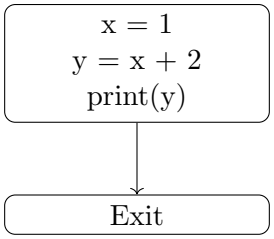
- The first statement of the program is a leader
- Any statement that is the target of a branch is a leader
- Any statement that immediately follows a branch or return statement is a leader

Each basic block begins with exactly one leader statement.

## 6 CFG for Straight-Line Code

**Code**

```
x = 1
y = x + 2
print(y)
```




All statements belong to a single basic block.

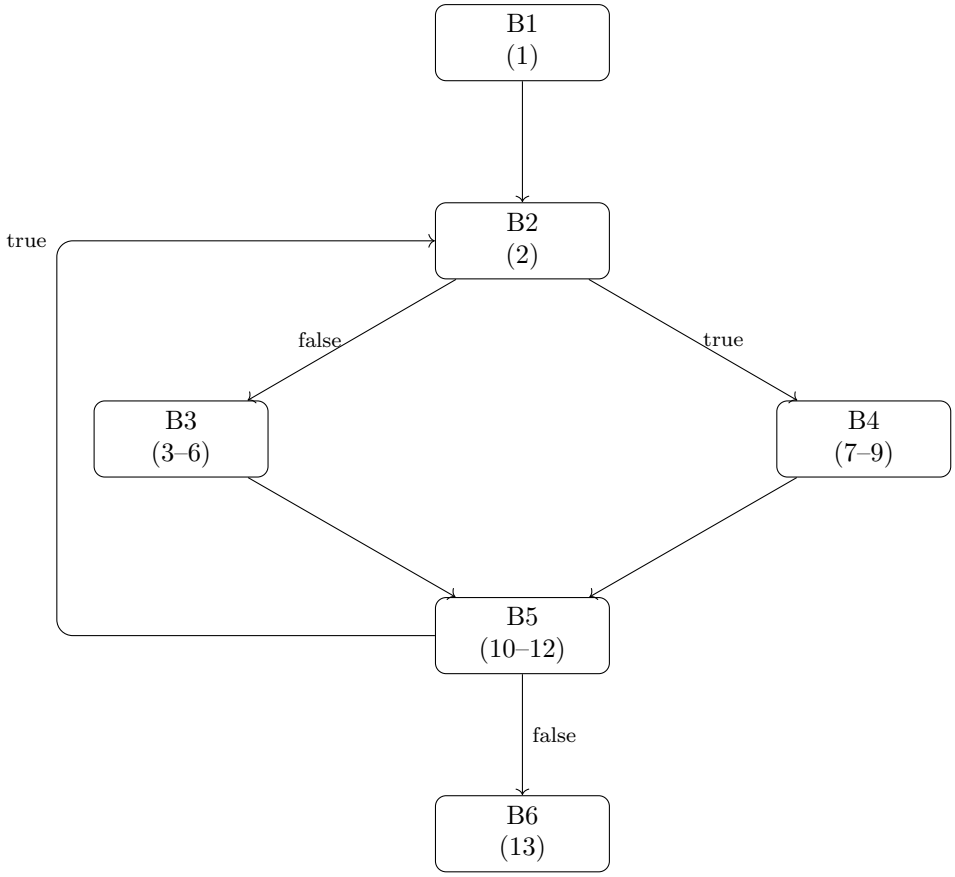
# 7 Example: Forming Basic Blocks

Consider the following intermediate-level code:

```
(1) initialize(...)

(2) if (theta > curr) goto (7)

(3) T = X + (Y >> i)
(4) Y = -(X >> i) + Y
(5) curr = curr - ctab[i]
(6) goto (10)

(7) T = X - (Y >> i)
(8) Y = (X >> i) + Y
(9) curr = curr + ctab[i]

(10) X = T
(11) i++
(12) if (i < STEPS) goto (2)

(13) return X, Y
```

The corresponding basic blocks are:

- B1: (1)
- B2: (2)
- B3: (3)–(6)
- B4: (7)–(9)
- B5: (10)–(12)
- B6: (13)




# 8 Conditional Statements

## 8.1 If Statement

**Code**

```
x = 0
print(x)
if x > 5:
    print('x is big')
else:
    print('x is small')
```

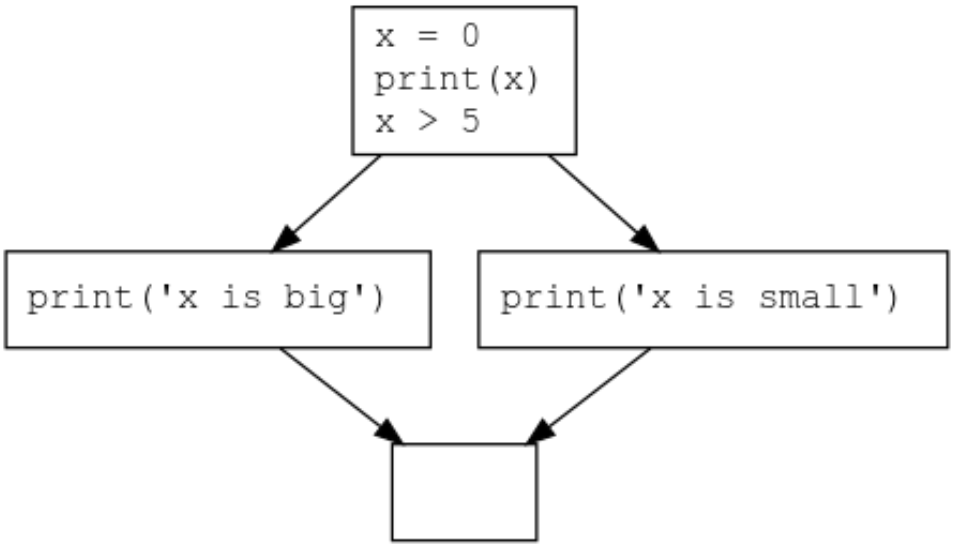



Figure 1: Control Flow Graph for an if–else statement showing branching based on the condition and merging after execution.

### 8.2 If–Else Statement

Both branches are mutually exclusive but merge after execution.

## 9 Loops

### 9.1 While Loop

**Code**

```
x = 0
while x < 10:
    print(x)
    x += 1

y = x + 3
```

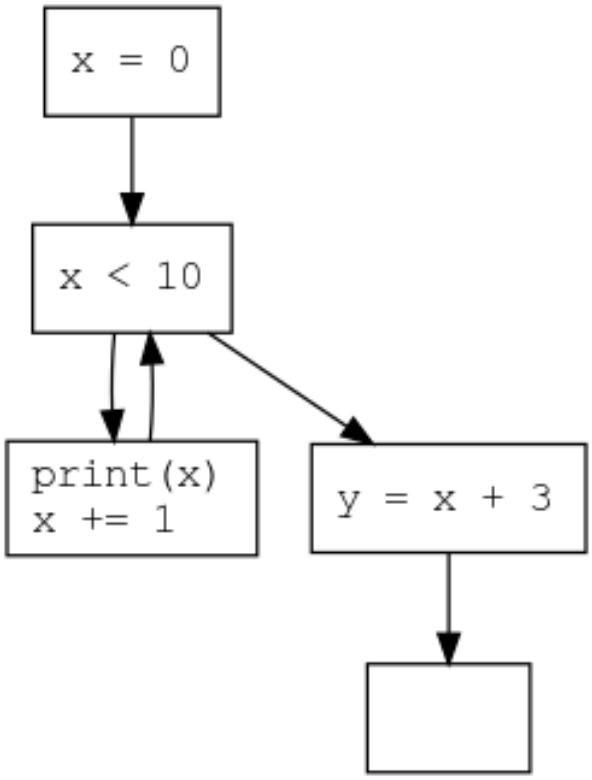



Figure 2: Control Flow Graph for a while loop illustrating the loop condition, loop body, and back edge forming a cycle.

Loops create **cycles** in the control flow graph.

### 9.2 Loop with Conditional

**Code**

```
x = 0
while x < 10:
    if x > 5:
        print(x)
    else:
        print('x less than 6')
    x += 1
```

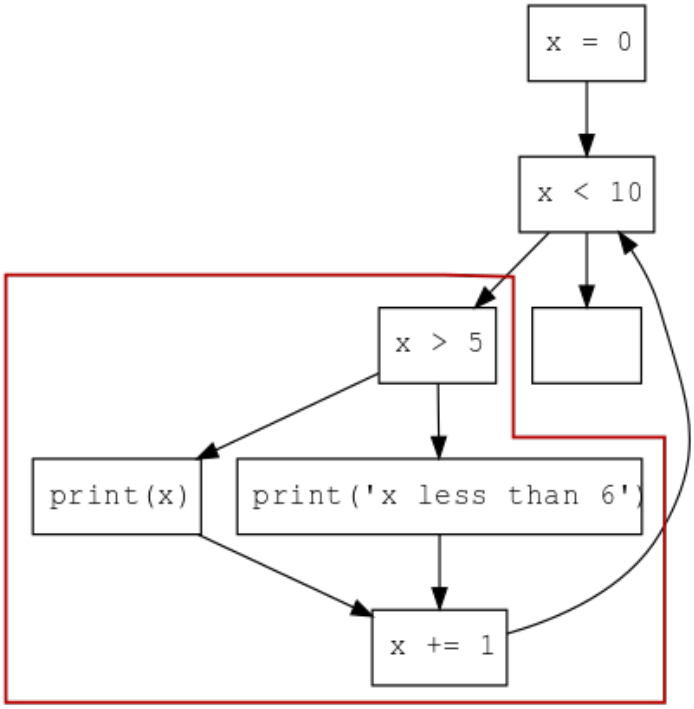



Figure 3: Control Flow Graph of a while loop containing an internal if–else conditional, showing nested branching within a loop.

## 10 Return Statements

**Code**

```
if x > 0:
    return 1
else:
    return -1
```

If all branches contain a `return`, the CFG has no continuation after the conditional.

### 10.1 Function with Conditional Return

**Code**

```
def f(x: int) -> None:
    if x > 10:
        return
    else:
        y = x + 1
    print(x * y)
```

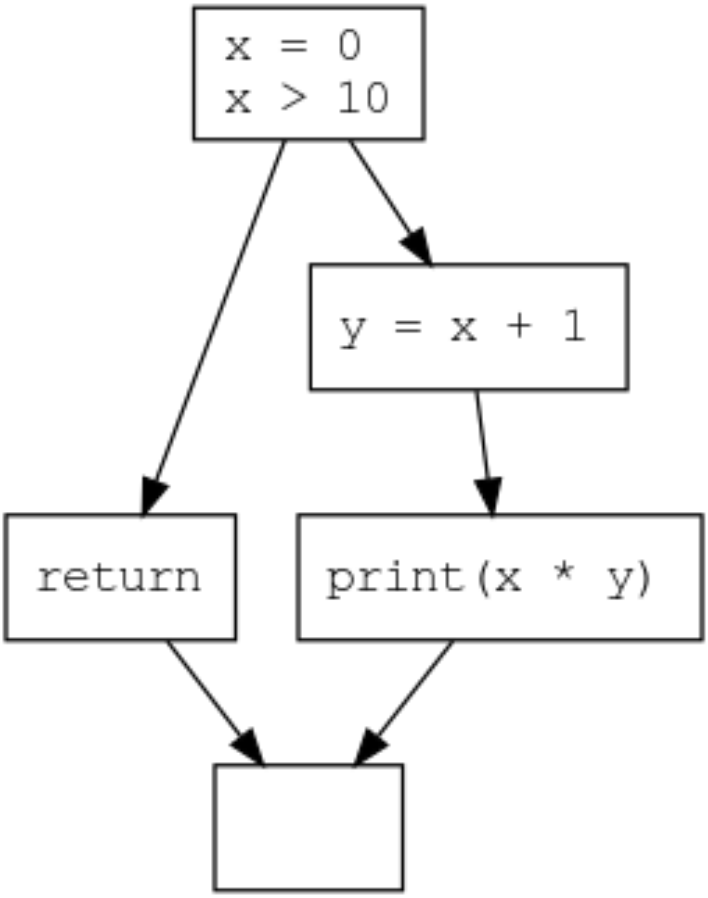



Figure 4: Control Flow Graph of a function with a conditional return, where one branch terminates execution early.

### 10.2 Loop with Early Return

**Code**

```
def has_even(numbers: list[int]) -> bool:
    """Return whether numbers has an even number."""
    i = 0
    while i < len(numbers):
        if numbers[i] % 2 == 0:
            return True
        else:
            return False
        i += 1
```

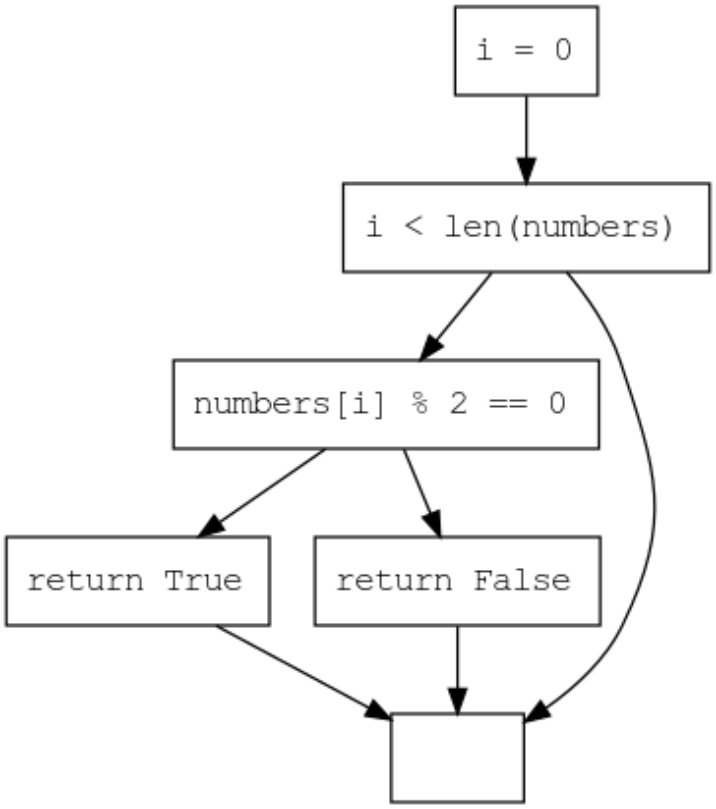



Figure 5: Control Flow Graph of a loop that executes at most once due to guaranteed return statements inside the loop.

### 10.3 Uninitialized Variable Risk

**Code**

```
x = 0
if is_special(x):
    x = 5
else:
    y = 5
print(x + y)
```

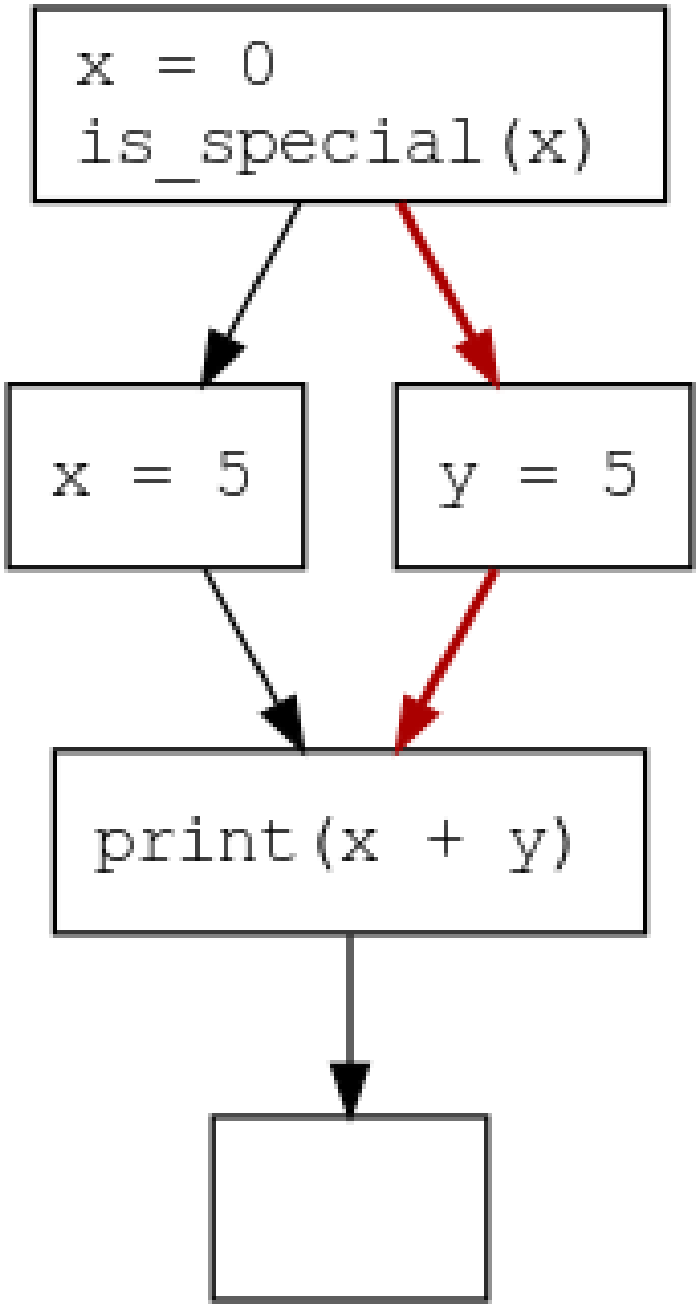



Figure 6: Control Flow Graph illustrating a potential uninitialized variable error caused by divergent execution paths.

## 11 Special Observations

### 11.1 Loops Executed At Most Once

If a loop body always returns, the CFG has no cycle and executes at most once.

### 11.2 Unreachable Code

Any node with no incoming edges (except entry) is unreachable.

## 12 Why CFGs Are Important

- Compiler optimization
- Dead code elimination
- Static analysis tools
- Program correctness checking

## 13 Exam Summary

- CFG is a directed graph
- Nodes represent basic blocks
- Edges represent control flow
- Conditionals cause branching
- Loops create cycles
- `return` leads to exit